DESIGNING DATA-INTENSIVE APPLICATIONS · VISUAL COMPANION

Chapter 2
Data Models and Query Languages

How relationship shape picks your model — and why emulating one model in another always hurts

Part I: Foundations · Illustrated · Interview-ready

Martin Kleppmann · Source: claude-skills / ch02-data-models-query-languages.md

Table of Contents

1. Core Idea

Every data model embodies assumptions about how data will be used — some operations become easy and fast, others awkward and slow. Applications layer models (app objects → general-purpose model like relational/document/graph → bytes on disk), and each layer hides the one below.

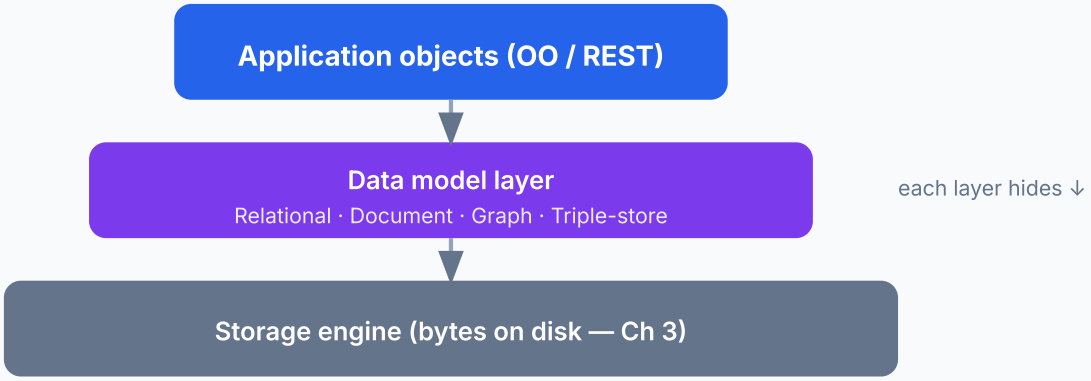

Figure 1.1 — Model layers: each tier hides complexity below; pick the model that matches your relationship structure.

The single most important discriminator between models is how they handle relationships: one-to-many (trees), many-to-one, and many-to-many. Pick the model that matches the relationship structure of your data, because emulating one model in another is always awkward.

2. Framework: Relationship-Driven Model Selection

Choose the data model by the dominant relationship type in your data.

Question	If yes → lean toward
Is the data tree-shaped and loaded whole?	**Document**
Do many-to-one / many-to-many relationships exist now, or will they as features grow?	**Relational**
Do queries need variable-length traversals across dense interconnection?	**Graph**

Failure mode: choosing documents for join-free simplicity, then features (entities-as-references, recommendations) introduce many-to-many relationships and you're rebuilding joins by hand — repeating IMS's 1970s hierarchical-model problems.

Evaluate models against future relationships, not just the initial shape. Data tends to become more interconnected over time.

3. Document vs Relational vs Graph

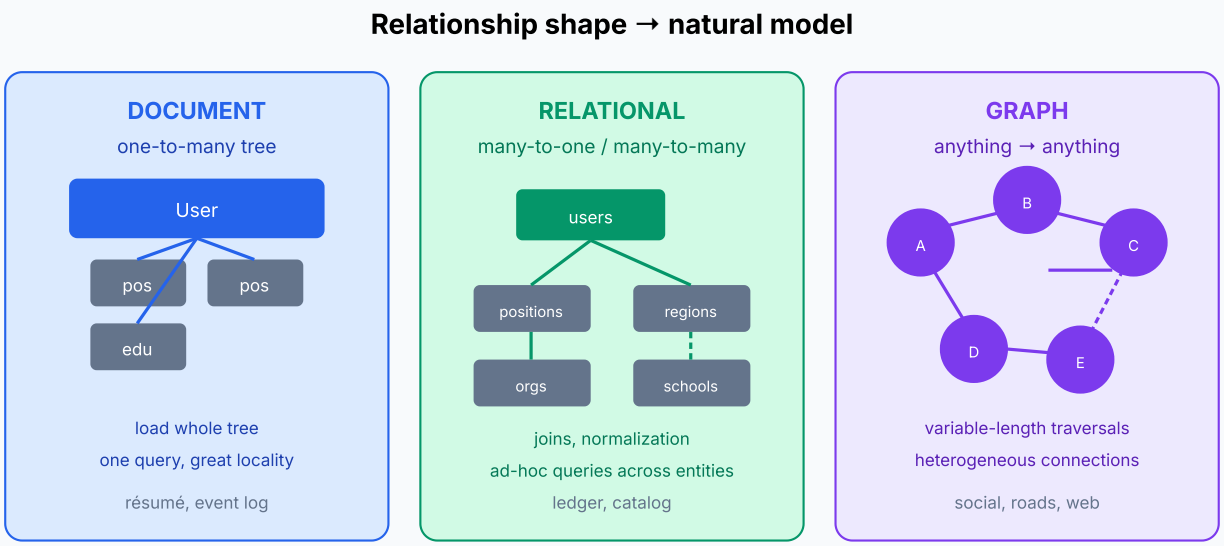

Figure 3.1 — Trees → document; joins/many-to-many → relational; dense interconnection → graph.

Model	When	How	Why / Failure
Document	Self-contained one-to-many tree; whole tree loaded at once; rare cross-document links	Embed nested arrays/objects (JSON/BSON); one read fetches profile	Great locality; weak joins. Shredding into tables is cumbersome; emulating joins in app code is slower than DB joins
Relational	Many-to-one and many-to-many common; need normalization and ad-hoc cross-entity queries	Tables + foreign keys; JOIN at query time	Flexible references; join machinery built-in. Impedance mismatch with OO objects (ORMs help, can't eliminate)
Graph	"Anything is potentially related to everything" — complex heterogeneous many-to-many	Vertices + labeled edges; declarative traversal (Cypher, SPARQL, Datalog)	Natural for unknown-depth paths. Not CODASYL redux: no fixed access paths, declarative queries

Convergence: PostgreSQL/DB2 add JSON columns; RethinkDB added joins. **Polyglot persistence** — use relational and non-relational side by side, each where it fits.

4. Schema-on-Read vs Schema-on-Write

Document databases are not "schemaless" — the reading code assumes an implicit schema, just unenforced.

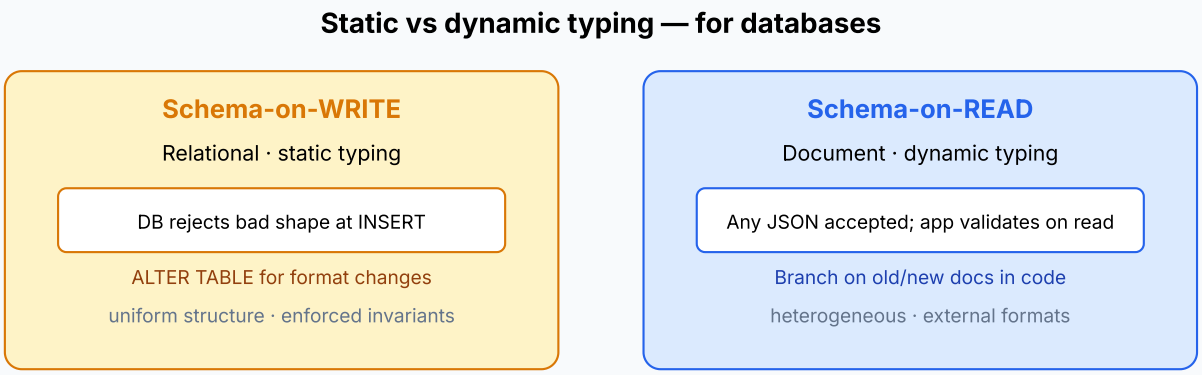

Figure 4.1 — Schema-on-write enforces at insert; schema-on-read assumes implicit structure in application code.

Approach	When to use	Trade-off
Schema-on-read	Heterogeneous data — many object types that can't each get a table; structure dictated by external systems you don't control	Flexible ingestion; schema lives implicitly in reading code (easy to miss inconsistencies)
Schema-on-write	All records share a structure; schema documents and enforces it	`ALTER TABLE` is milliseconds on most DBs (MySQL copies whole table — slow). Full-table `UPDATE` is slow anywhere; backfill lazily at read time instead

Schema-on-read = dynamic typing; schema-on-write = static typing. Neither is universally right — it's a tradeoff, not a winner.

5. Declarative vs Imperative Querying

Declare the *pattern* of data you want (conditions, transformations), not the algorithm. The query optimizer picks indexes, join methods, and execution order — the "access path" chosen automatically instead of hand-coded by the developer.

Style	You specify	Engine handles	Historical note
Declarative	WHAT (SQL, Cypher, SPARQL, CSS selectors)	Indexes, join order, parallelism, storage layout changes	Order-independent → optimizer can reorganize without breaking queries
Imperative	HOW — step-by-step navigation (CODASYL access paths)	Nothing — developer pins ordering	CODASYL's fatal flaw: inflexible when storage or APIs change
MapReduce	Two coordinated functions (map + reduce)	Framework calls them repeatedly	Neither fully declarative nor imperative — MongoDB later added aggregation pipeline ("accidentally reinventing SQL")

Build the optimizer once, every application benefits. Imperative code that depends on record ordering blocks storage reorganization, optimization, and parallelism — same lesson as CSS selectors vs imperative DOM manipulation.

Graph vs SQL for variable-length traversal: "Find people who emigrated from the US to Europe" — ~4 lines in Cypher, ~29 lines of SQL with `WITH RECURSIVE` CTEs. Unknown-depth joins are native to graph languages; clumsy in SQL.

6. Key Concepts

Term	Definition
Impedance mismatch	Awkward translation between OO application objects and relational tables/rows/columns; ORMs reduce boilerplate but can't eliminate it
Normalization	Store human-meaningful values once, reference by ID (IDs never need to change). Denormalization duplicates for read convenience — write overhead and inconsistency risk
Locality	Document as one contiguous string → one read fetches all — only wins if you need most of it. Updates rewrite whole document → keep documents small
Hierarchical model (IMS, 1968)	Tree of nested records — essentially JSON documents circa 1968; one-to-many well, many-to-many badly, no joins
CODASYL / network model	1970s: records with multiple parents via pointers; querying = manual access-path navigation — inflexible imperative code
Property graph	Vertices and edges with ID + key-value properties; edges labeled with tail/head (Neo4j, Cypher)
Triple-store	All data as (subject, predicate, object); equivalent to property graph, different vocabulary (Datomic, SPARQL, RDF/Turtle)
Datalog	Rule-based query language (Prolog subset); facts as predicate(subject, object); rules compose and recurse — foundation graph languages build on
Polyglot persistence	Relational and non-relational stores used side by side, each where it fits
MapReduce querying	Pure map/reduce snippets the framework calls repeatedly — harder to write and optimize than declarative pipelines

7. Mental Models

1. Relationship → model

Document for self-contained trees loaded whole. **Relational** when many-to-one/many-to-many need joins. **Graph** when highly interconnected — "for highly interconnected data, the document model is awkward, the relational model acceptable, graph models the most natural."

2. Schema timing

Schema-on-read for heterogeneous or externally controlled records. **Schema-on-write** when uniform structure should be enforced — static vs dynamic typing for databases.

3. IDs over strings

Whenever a value might change or be shared: IDs have no human meaning, so they never need updating. Duplicated human-readable values ("Greater Seattle Area") must be updated everywhere — the case for normalization.

4. Graph ≠ CODASYL

No nesting schema (any vertex links to any vertex), direct access by ID/index instead of access paths, no maintained ordering, declarative queries (Cypher/SPARQL) instead of imperative traversal.

8. Anti-Patterns

Anti-pattern	Why it fails
Denormalizing into documents to avoid joins, then hand-rolling join logic in app code	Slower and more complex than database joins; you rebuild IMS problems
Treating "schemaless" as no schema	Schema moves into reading code — implicit and unenforced
Storing mutable human-meaningful strings duplicated across records	Every copy must be updated on change; use ID references instead
Large, growing documents	Engines load and rewrite whole documents — size growth destroys locality advantage
Imperative query code depending on record ordering	Blocks storage reorganization, optimization, and parallelism
Deeply nested documents	Can't reference nested item directly — only via access-path-like positions ("second item in positions of user 251")

9. Worked Example: LinkedIn Résumé (Bill Gates, user_id 251)

Phase 1 — Tree-shaped profile favors documents

Relational form: a `users` row (first_name, last_name, summary, region_id, industry_id) plus separate `positions`, `education`, `contact_info` tables with `user_id` foreign keys, and normalized `regions` / `industries` lookup tables. Fetching a profile needs a multi-way join or several queries.

Document form: one JSON document with `positions` and `education` as embedded arrays — the one-to-many tree is explicit, locality is great, one query fetches everything. **This favors documents.**

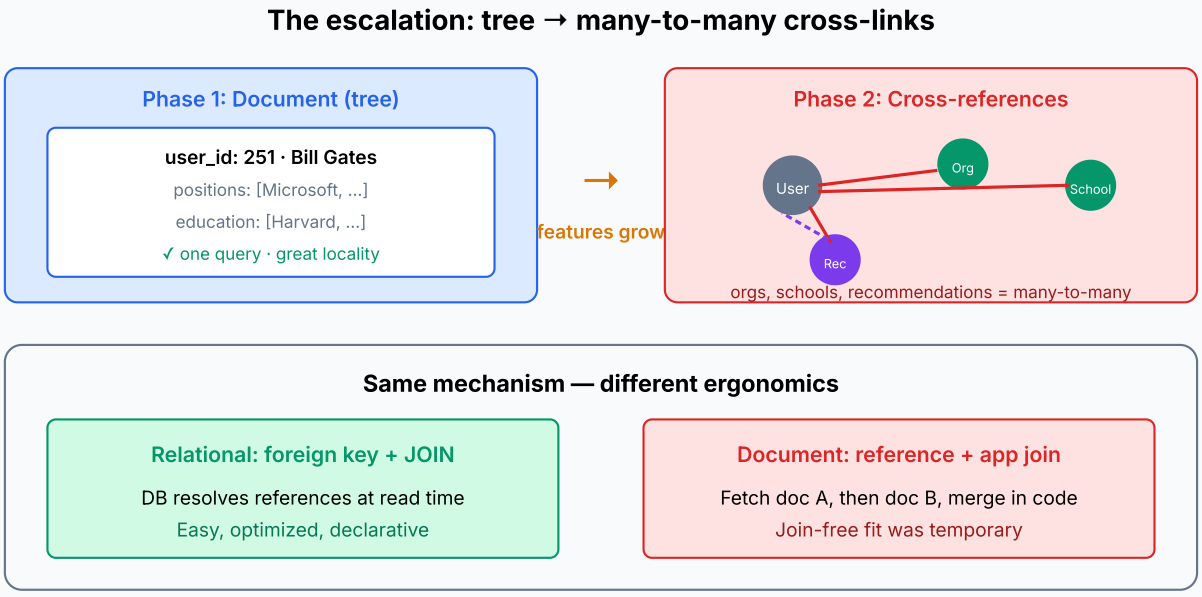

Figure 9.1 — LinkedIn résumé: documents win for the initial tree; feature growth introduces references that relational DBs join easily.

Phase 2 — The escalation breaks the tree

- **Organizations and schools as first-class entities** — job entries must *reference* org entities (with pages/logos), not embed strings.
- **Recommendations** — show recommender's current name and photo; must reference author's profile (photo update propagates everywhere).
- These are **many-to-many relationships**: dotted-box "document" parts remain, but cross-links require references resolved by joins at read time.

Interconnectedness grows with features. In both models the mechanism is the same — a foreign key (relational) or document reference (document) — but relational databases make those joins easy while document databases push the work into application code.

Query-language contrast

```
-- Cypher (~4 lines): variable-length traversal native
(person) -[:BORN_IN]→ () -[:WITHIN*0..]→ (us:Location {name:'United States'})
(person) -[:LIVES_IN]→ () -[:WITHIN*0..]→ (eu:Location {type:'Continent', name:'Eu

-- SQL (~29 lines): WITH RECURSIVE CTEs for unknown-depth joins
```

10. Problems Each Mechanism Solves

Mechanism	Problem it solves	Example
Document model for self-contained one-to-many tree	Structure always read/written whole; locality matters	Search autocomplete (trie serialized into a document store)
Graph model — "anything can relate to anything"	Dense many-to-many where joins would be unbounded-depth	News feed system (social graph for friend IDs); Google Maps (road network as nodes/edges for A*)
Relational model chosen deliberately over NoSQL	Strict invariant (sums to zero) needing real joins/constraints, not app-level enforcement	Payment system (double-entry ledger on ACID SQL)

11. Connects To (Cross-Chapter)

Chapter	Connection	Interview soundbite
Ch 3 Storage	How these models are implemented in storage engines; locality on disk	"Document locality on disk depends on engine — one read if contiguous."
Ch 4 Encoding	Encoding formats, JSON's problems, schemas and schema evolution	"Schema-on-read doesn't skip evolution — it moves validation to the app."
Ch 5 & Ch 7	Fault tolerance and concurrency differences between relational and document stores	"Model choice affects how replication and transactions behave."
Ch 10 Batch	MapReduce in full; graph processing (Pregel); SQL as MapReduce pipeline	"MapReduce sits between declarative SQL and imperative loops."
Part III	Systematic treatment of caching, denormalization, and derived data	"Denormalization trades write complexity for read speed — Part III deep dive."

12. Key Takeaways

1. **Data models shape how you think about the problem;** each layer's model hides the complexity below.
2. **Relationship structure decides the model:** trees → document, joins/many-to-many → relational, dense interconnection → graph. All three are widely used; emulating one in another is awkward.
3. **Document databases are the hierarchical model (IMS, 1968) reborn** — great locality and schema flexibility, weak joins and many-to-many. History's fix was the relational model; know why before opting out of it.
4. **Schema-on-read vs schema-on-write** is a tradeoff, not a right answer — heterogeneous data favors the former, uniform data the latter.
5. **Declarative languages** (SQL, Cypher, SPARQL, CSS) win because they hide implementation, enable automatic optimization, and parallelize; imperative access-path code (CODASYL) is what they replaced.
6. **Data becomes more interconnected as applications grow** — evaluate models against future relationships, not just the initial shape.
7. **Relational and document databases are converging** (JSON in PostgreSQL/DB2, joins in RethinkDB); a hybrid is a good route — use the combination that fits.

13. Junior SWE Checkpoints

Q1: What's the single most important discriminator between data models?

How they handle **relationships** — one-to-many (trees), many-to-one, and many-to-many. Pick the model matching your dominant relationship shape; emulating one in another is always awkward.

Q2: When does a LinkedIn-style résumé favor documents over relational?

When the profile is a **self-contained one-to-many tree** (positions, education embedded) loaded whole — one query, great locality. Relational needs multi-way joins for the same fetch.

Q3: Why does the document model break down as features grow?

Organizations, schools, and recommendations introduce **many-to-many cross-references** (org pages, recommender photos that must stay current). Same mechanism as foreign keys, but document DBs push join resolution into application code.

Q4: "Schemaless" document DB — is there really no schema?

No. Schema-on-read: the reading code assumes an implicit schema, just unenforced — like dynamic typing. The schema moved into app code, where it's invisible and inconsistent.

Q5: Normalization vs denormalization — when use IDs?

Use **IDs over human-meaningful strings** whenever a value might change or be shared. IDs have no meaning → never need updating. Duplicated strings ("Greater Seattle Area") must be updated everywhere on change.

Q6: Why did declarative query languages replace CODASYL?

Declarative (SQL, Cypher) specifies **what**, not **how**. Optimizer picks indexes, join order, parallelism. Imperative access-path code pins ordering — blocks storage changes and can't be optimized globally.

Q7: Graph databases vs CODASYL network model — same thing?

No. Modern graphs: no nesting schema, direct ID/index access (not fixed access paths), no maintained ordering, **declarative** queries (Cypher/SPARQL). CODASYL required imperative pointer navigation.

Glossary

Term	Definition
Impedance mismatch	OO objects vs relational tables — ORMs help, can't eliminate
Normalization	Store values once, reference by ID; avoid duplicate mutable strings
Locality	One read fetches whole document — wins only if you need most of it
Schema-on-read	Implicit schema enforced in application code at read time
Schema-on-write	Explicit schema enforced by DB at write time
Property graph	Vertices + labeled edges with properties (Neo4j, Cypher)
Triple-store	(subject, predicate, object) statements — SPARQL, RDF
Datalog	Rule-based queries with recursive composition
Polyglot persistence	Multiple DB types side by side, each where it fits
CODASYL	1970s network model with imperative access-path navigation
Hierarchical model (IMS)	1968 tree-of-records — ancestor of document databases
MapReduce querying	Map + reduce functions — MongoDB later added declarative aggregation